

Oracle Zero-Trust Packet Routing Policy Enforcement Language

January 27, 2025

The Oracle Zero-Trust Packet Routing Policy Enforcement Language (ZPR PEL) is a high-level language for describing network access policies within a Virtual Cloud Network (VCN), such that a policy can be enforced by checking each proposed network connection at the source (that is, initiating) endpoint and at the target (that is, receiving) endpoint.

This serves two purposes: on the one hand, to provide a layer of access policy enforcement that is independent of, and can backstop, finer-grained access policies, such as Oracle Identity and Access Management (IAM) policies, and on the other hand to serve as a target language for a higher-level policy language—such as ZPL, the ZPR Policy Language for Zero-Trust Packet Routing—that provides a stricter separation of concerns between network configuration and network security.

A ZPR PEL policy associated with a VCN consists of a set of `allow` statements, each of which specifies conditions that, if satisfied for a proposed network connection, permit a source endpoint to establish communication with a target endpoint. Oracle ZPR PEL Version 1 assumes that every endpoint corresponds to some IP address. An endpoint not within the VCN must be described by a specific IP address or by a CIDR block of IP addresses, but an endpoint within the VCN must be described abstractly in terms of associated security attributes maintained by a trusted source. At least one of the two endpoints mentioned in an `allow` statement must be within the VCN enforcing the policy. Therefore there are, in effect, three different types of `allow` statements: (1) those governing *internal* connections, between two endpoints within the VCN; (2) those governing *outgoing* connections, from endpoints within the VCN to endpoints outside the VCN; and (3) those governing *incoming* connections, from endpoints outside the VCN to endpoints within the VCN. Source endpoints enforce types (1) and (2); target endpoints enforce types (1) and (3).

This document describes Oracle ZPR PEL Version 1 and its intended applications in detail.

1 Technical Overview

For the purposes of this document, a VCN consists of a set of *endpoints* and some means of making *connections* between pairs of endpoints over the course of time. A connection is established when a first endpoint (the *source* endpoint, sometimes also called the *client* endpoint) *proposes* a connection to a second (possibly the same) endpoint, and that second endpoint (the *target* endpoint, sometimes also called the *service* endpoint) *accepts* the connection. A connection may persist for some time, and may eventually be dropped (disestablished), but such actions are beyond the scope of this document, which addresses only the question of whether a proposed connection is allowed by the policy of the VCN.

Oracle ZPR PEL Version 1 assumes that every endpoint is a VNIC, and vice versa, and that every

connection is an IP connection. It also assumes that some trusted source maintains and associates *security attributes* for every endpoint in the VCN and also for the VCN itself, where every security attribute for an endpoint or VCN is identified by a *key* (a name) and may have an associated *value*. An endpoint may have any number of associated security attributes (even though ZPR PEL Version 1 allows at most one security attribute to be specified for each endpoint mentioned in a policy statement). The trusted source may organize the names used as keys into *namespaces*, and may also provide a default namespace (which allows a key to be mentioned in a policy statement without explicitly mentioning the namespace to which the key belongs). How this trusted source is implemented, including what namespaces may be available and whether there is a default namespace, is beyond the scope of this document, except for the assumption that when a connection is proposed, the actual security attributes of the source endpoint and the target endpoint are made available to help determine whether the proposed connection is allowed by the policy of the VCN.

When a specific connection is proposed, additional “ordinary” (non-security) attributes that describe the proposed connection become temporarily associated with the endpoints for the purpose of trying to match a policy statement. Examples of such attributes are the TCP protocol to be used for the connection and the port number that endpoint will use for the connection.

The policy for a VCN consists of a set of policy statements. The manner in which statements are gathered into a set and associated with a VCN so that they will be enforced is beyond the scope of the document.

Every **allow** statement specifies positive permission. A specific proposed connection is *permitted* if and only if it *matches* at least one policy statement in the set, thus affirming positive permission for that connection.

Oracle ZPR PEL Version 1 does not contain negative permission statements (such as **deny** statements).

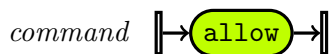
Here are some general, informal comments about how each policy statement is written:

- All keywords, IP addresses, CIDR blocks, names, and strings—including names and strings used as values or simple values—are case-insensitive (see Section 4). Each letter may be either uppercase or lowercase, freely. Whenever a name, string, or value specified within a ZPR PEL policy statement is compared to a name, string, or value obtained from another source, such as a trusted source for security attributes, all the letters in each that have lowercase equivalents are converted to those lowercase equivalents before the comparison operation is performed.
- Whitespace (such as spaces, tabs, and/or newlines) is necessary to separate a keyword, name, or number from an immediately following keyword, name, or number; and whitespace is significant and used as-is within strings; but otherwise whitespace may be used or omitted freely.

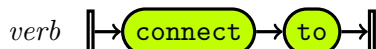
2 Syntax of Policy Statements



Every policy statement has a *command*, preceded by a *network scope* clause (see Section 2.1) and followed by a *source endpoint* clause, a *verb* (preceded by the keyword **to**), and a *target endpoint* clause, in that order.



In Oracle ZPR PEL Version 1, the only available command is **allow**.



In Oracle ZPR PEL Version 1, the only available verb is `connect to`.



The *source endpoint* in a policy statement is specified as an *endpoint* (see Section 2.2).



The *target endpoint* in a policy statement is specified as an *endpoint* (see Section 2.2).

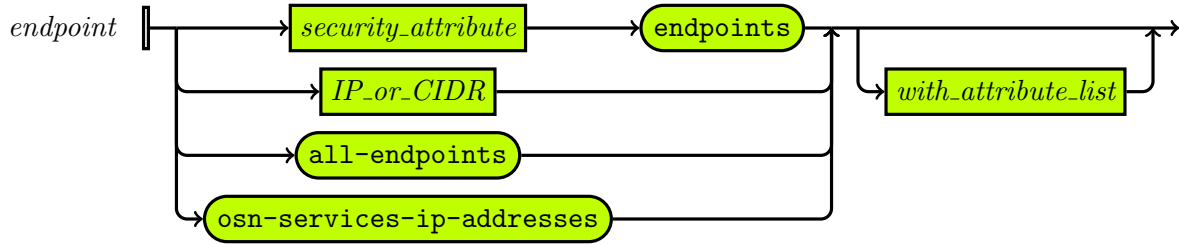
2.1 Identifying the relevant VCN (“network scope”)



A *network scope clause* indicates which security attribute the enforcing VCN must have if a proposed connection is to match.

In Oracle ZPR PEL Version 1, exactly one security attribute must be specified for the enforcing VCN.

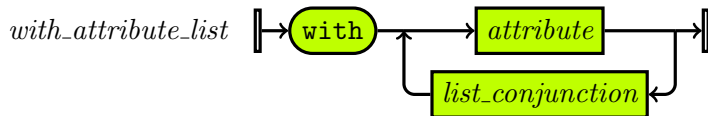
2.2 Endpoints



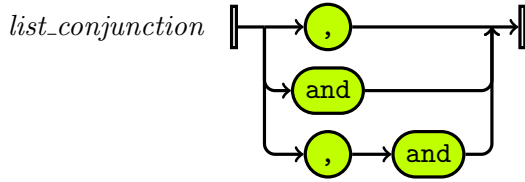
The *source endpoint* clause and the *target endpoint* clause of a policy statement have the same general *endpoint* clause syntax: a *basic endpoint* clause, which provides four choices, optionally followed by a *with attribute list* clause that lists additional ordinary (non-security) attributes that the endpoint must have in order to match the policy statement.

A *basic endpoint* is specified in one of four ways:

- The keyword `endpoints`, preceded by a *security attribute* clause (see Section 2.3).
- A specific IP address or CIDR block.
- The keyword `all-endpoints`.
- The keyword `osn-services-ip-addresses`.



A *with attribute list* clause consists of the keyword `with` followed by one or more (ordinary) *attribute* clauses (see Section 2.4), where consecutive such clauses are separated by a *list conjunction*.



Each *list conjunction* may be either ‘,’ or ‘and’ or ‘, and’, freely.

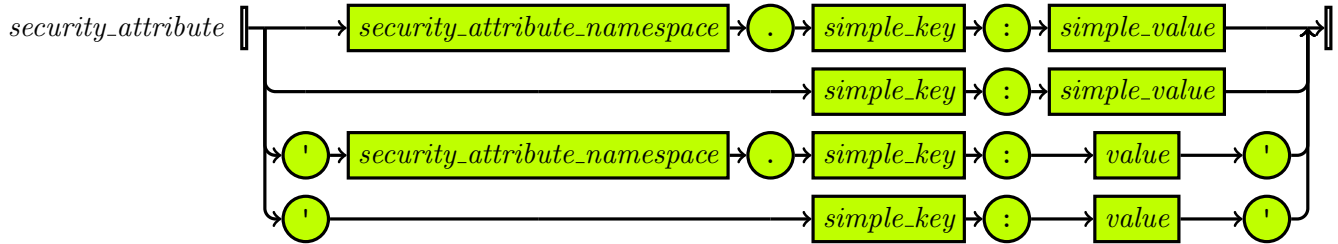
In order to match the policy statement, all of the *attribute* clauses must be matched by ordinary (non-security) attributes of the endpoint.

The order in which *attribute* clauses appear within a *with attribute list* clause does not matter. For example, the following *with attribute list* clauses all mean the same thing:

```

with protocol='icmp', protocol.icmp.type='12', protocol.icmp.code='2'
with protocol='icmp' and protocol.icmp.type='12' and protocol.icmp.code='2'
with protocol='icmp', protocol.icmp.type='12', and protocol.icmp.code='2'
with protocol.icmp.type='12' and protocol.icmp.code='2' and protocol='icmp'
  
```

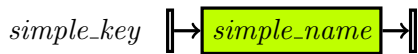
2.3 Security attributes



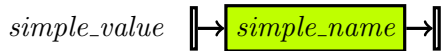
A security attribute is a key-value pair. A policy statement specifies a required security attribute for an endpoint by mentioning the required key, a colon, and the required value. The key may be specified as a simple name or as a name within a specified security attribute namespace, which is written before the key with a separating ‘.’ (period) character. If the value is not a simple value, then the entire security attribute clause must include a pair of surrounding single-quote characters.



A security attribute namespace is specified by a simple name.



A security attribute key is specified by a simple name.



A security attribute value may specified by a simple name. As already mentioned above, if the value is not simple then surrounding single-quote characters ust be used.



A simple name may be any nonempty sequence of ASCII letters, digits, hyphens, and underscores. Examples of simple names include `color`, `marketing-department`, `411`, and `user_name`.

value $\parallel \rightarrow \boxed{\text{string}} \rightarrow \parallel$

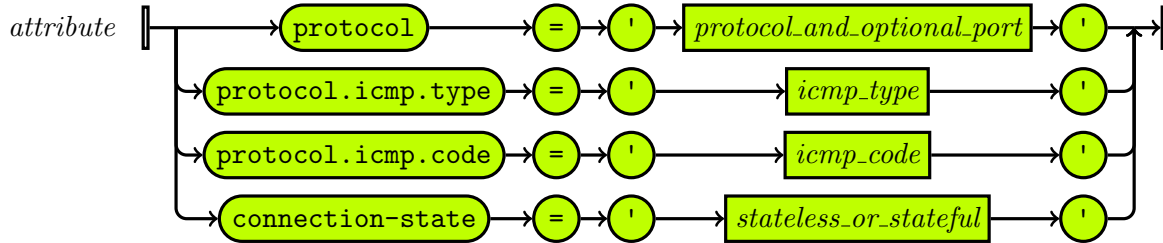
A value that is not simple may be any string (see below).

string $\parallel \rightarrow \boxed{\text{any sequence of characters from the set of allowed string characters}} \rightarrow \parallel$

If there is a need to mention a single-quote character within a string, it should be doubled. For example, if the value of a security attribute with key `user_name` needs to be `0'Malley`, the security attribute should be specified in a policy statement as `user_name:0''Malley`.

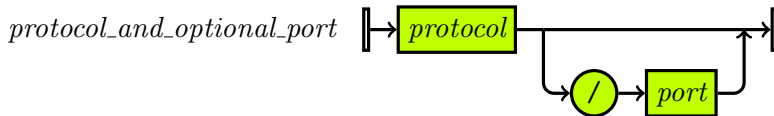
2.4 Ordinary attributes

In addition to requiring an endpoint to have a specific security attribute, a policy statement may also require the endpoint to have one or more ordinary (non-security) attributes. These are specified in a *with attribute list* associated with the endpoint (see Section 2.2).



Oracle ZPR PEL Version 1 supports the specification of four ordinary attributes; their names are `protocol`, `protocol.icmp.type`, `protocol.icmp.code`, and `connection-state`. While period characters '.' are not permitted in the name of the keys of security attributes, to avoid ambiguity about where the name of a namespace ends and the name of a key begins (see Section 2.3), period characters are used in the names of some ordinary attributes.

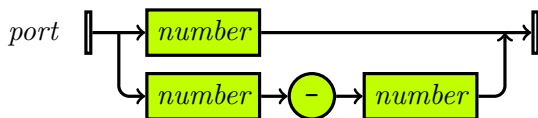
The specified value for an ordinary attribute must always be surrounded by single-quote characters, even if the value has the form of a simple value. The single-quote characters surround only the value of an ordinary attribute; this is different from the use of single-quote characters to surround both the key and the value of a specified security attribute (see Section 2.3).



The value for the `protocol` attribute may be either a protocol, such as `http`, or a protocol followed by a slash character and a port number, for example `tcp/443`.

protocol $\parallel \rightarrow \boxed{\text{string}} \rightarrow \parallel$

A *protocol* is written as a string. See Section 3.1 for a detailed description of what strings are valid *protocol* specifications.



A *port* is written either as a single number or as a pair of numbers separated by a hyphen character

‘-’ (representing a range of allowable port numbers). See Section 3.1 for a detailed description of what numbers are valid in *port* specifications.

icmp_type $\parallel \rightarrow$ value $\rightarrow \parallel$

See Section 3.1 for a detailed description of what values are valid *icmp_type* specifications.

icmp_code $\parallel \rightarrow$ value $\rightarrow \parallel$

See Section 3.1 for a detailed description of what values are valid *icmp_code* specifications.

stateless_or_stateful $\parallel \rightarrow$ stateless $\rightarrow \parallel$
stateful $\rightarrow \parallel$

The specified value for a **connection-state** attribute may be either **stateless** or **stateful**. If no **connection-state** attribute is specified explicitly, the connection will be required to be stateful (see Section 3.3).

number $\parallel \rightarrow$ any nonempty sequence of ASCII digits $\rightarrow \parallel$

Oracle ZPR PEL Version 1 supports decimal numbers such as 0 and 443, but not hexadecimal notations such as 0xf3.

3 Matching Policy Statements to Specific Connections

Call a VCN that is enforcing a set of ZPR PEL policy statements the “enforcing VCN.” Whenever there is an attempt to create an actual network connection between a source endpoint and a target endpoint, at least one of which is within the enforcing VCN, the following information for each endpoint is assumed to be available to the enforcement mechanism:

- The enforcing VCN.
- The security attributes of the enforcing VCN.
- The protocol to be used for the network connection.
- If the protocol is to be ICMP or IPv6-ICMP, the ICMP type and ICMP code.
- Whether the network connection is to be stateful or stateless.
- The IP address of the endpoint.
- The port number to be used by that endpoint for the network connection.
- Whether the endpoint is within the enforcing VCN.
- If the endpoint is within the enforcing VCN, the security attributes of that endpoint.

The first five items of information are common to both endpoints; the last four items may be different for each endpoint.

This information is then used to determine whether the information for the proposed actual connection *matches* any ZPR PEL **allow** policy statement in the set. If the information matches at least one **allow** policy statement in the set, the connection is established; if the information does not match any policy statement in the set, the attempt is rejected and no connection is established.

This structure is designed so that the enforcement mechanism can be implemented by checking the policies only at endpoints within the VCN, as follows:

- Before attempting to establish a connection, a source endpoint must first verify that the proposed

- connection matches at least one **allow** statement of type (1) or type (2)—that is, internal or outgoing.
- Before accepting a connection, a target endpoint must first verify that the proposed connection matches at least one **allow** statement of type (1) or type (3)—that is, internal or incoming.

3.1 Matching a policy statement

The information for a proposed actual connection *matches* a policy statement if all of the following are true:

- The policy statement is well-formed.
- The information for the source and target endpoints matches the network scope clause in the policy statement (see Section 3.2).
- The information for the source endpoint matches the source endpoint clause in the policy statement (see Section 3.3).
- The information for the target endpoint matches the target endpoint clause in the policy statement (see Section 3.3).

Note that because some information about a proposed connection, such as the protocol to be used for the connection, is common to both endpoints, it is possible to write a policy statement that no proposed connection can possibly match; this can happen if each endpoint clause has an attribute clause and the values specified in the two attribute clauses are not compatible. For example, no proposed connection could possibly match a policy statement in which the source endpoint clause contains an attribute clause with `protocol='tcp'` and the target endpoint clause has an attribute clause with `protocol='icmp'`.

A policy statement is *well-formed* if all of the following are true:

- The statement obeys the syntax specified by the ZPR PEL BNF grammar (see Section 4).
- At least one of the following is true:
 - The source endpoint clause contains a security attribute clause.
 - The target endpoint clause contains a security attribute clause.
- The source endpoint clause is well-formed.
- The target endpoint clause is well-formed.

An endpoint clause is *well-formed* if all of the following are true:

- If the endpoint clause contains an *IP_or_CIDR* clause, then the *IP_or_CIDR* clause is well-formed.
- If a `with_attribute_list` is present, then no two distinct attribute clauses have the same name.
- If an attribute with name `protocol` is present, then the corresponding value is a well-formed protocol value.
- If an attribute with name `protocol.icmp.type` is present, then the corresponding value is a decimal integer in the range 0–255, inclusive.
- If an attribute with name `protocol.icmp.code` is present, then the corresponding value is a decimal integer in the range 0–15, inclusive.
- If an attribute with name `protocol.icmp.type` is present, then an ICMP protocol attribute is present.
- If an attribute with name `protocol.icmp.code` is present, then an ICMP protocol attribute is present.
- If an attribute with name `connection-state` is present, then the corresponding value is either `stateless` or `stateful`.

An *IP_or_CIDR* clause is well-formed if any of the following is true:

- It is an IPv4 address expressed in standard “dot-decimal” notation as four decimal numbers, each in the range 0–255, inclusive, separated by period characters ‘.’.

- It is an IPv4 CIDR block expressed as an IPv4 address followed by a slash character ‘/’ and a decimal integer (call it ‘*k*’) in the range 0–32, inclusive, such that all bits in the IPv4 address other than the leftmost *k* bits are 0.
- It is an IPv6 address expressed in standard “colon-hexadecimal” notation, either in full form as eight hexadecimal integers, each having four digits, separated by colon characters ‘:’, or in any of the customary abbreviated forms allowed by omission of redundant leading zeroes in each group and/or omission of one sequence of all-zero groups and the colon characters that separate them.
- It is an IPv6 CIDR block expressed as an IPv6 address followed by a slash character ‘/’ and a decimal integer (call it ‘*k*’) in the range 0–128, inclusive, such that all bits in the IPv6 address other than the leftmost *k* bits are 0.

A value is a well-formed protocol value if any of the following is true:

- The value is a decimal integer in the range 0–255, inclusive.
- The value is the four characters `tcp/` followed by a character sequence that is a well-formed port specification.
- The value is the four characters `udp/` followed by a character sequence that is a well-formed port specification.
- The value is `icmp`.
- The value is `ipv6-icmp`.

A character sequence is a well-formed port specification if any of the following is true:

- It is a decimal integer in the range 1–65535.
- It is a first decimal integer followed by a hyphen – followed by a second decimal integer, where each decimal integer is in the range 1–65535 and the second decimal integer is not smaller than the first decimal integer.

An ICMP protocol attribute is present if all of the following are true:

- An attribute with name `protocol` is present.
- The corresponding value for that attribute is one of the following:
 - `icmp`
 - `1`
 - `ipv6-icmp`
 - `58`

3.2 Matching a network scope clause

The information for the source and target endpoints *matches* the network scope clause in the policy statement if all of the following is true:

- The enforcing VCN has a security attribute that matches the security attribute clause in the network scope clause of the policy statement.
- At least one of the following is true:
 - The source endpoint is within the VCN that is enforcing the set of policy statements.
 - The target endpoint is within the VCN that is enforcing the set of policy statements.

3.3 Matching an endpoint clause

The information for an endpoint matches an endpoint clause in the policy statement if all of the following are true:

- If the endpoint is within the enforcing VCN, then all of the following are true:

- The endpoint clause contains a security attribute clause.
- The endpoint has a security attribute that matches the security attribute clause in the endpoint clause.
- If the endpoint is not within the enforcing VCN, then all of the following are true:
 - The endpoint clause does not contain a security attribute clause.
 - If the endpoint clause contains an *IP_or_CIDR* clause, then the IP address of the endpoint matches the *IP_or_CIDR* clause (see Section 3.4).
 - If the endpoint clause contains **all-endpoints**, then the IP address of the endpoint matches either 0.0.0.0/0 or 0::0/0.
 - If the endpoint clause contains **osn-services-ip-addresses**, then the IP address of the endpoint matches any one of the set of the IPv4 and IPv6 address ranges used by Oracle services.
- If the endpoint clause contains a **with_attribute_list** clause, then the information for the endpoint matches every attribute clause (see Section 3.5) listed in the **with_attribute_list** clause.
- If the endpoint clause contains a **with_attribute_list** clause, but no attribute clause listed in that **with_attribute_list** clause has the name **connection-state**, then the network connection is to be stateful.
- If the endpoint clause does not contains a **with_attribute_list** clause, then the network connection is to be stateful.

3.4 Matching an *IP_or_CIDR* clause

The IP address of an endpoint matches an *IP_or_CIDR* clause if (and only if) all of the following are true:

- The endpoint is not within the VCN.
- One of the following is true:
 - The *IP_or_CIDR* clause is an IPv4 address that equals the IP address of the endpoint.
 - The *IP_or_CIDR* clause is an IPv6 address that equals the IP address of the endpoint.
 - The *IP_or_CIDR* clause is an IPv4 CIDR block that contains the IP address of the endpoint.
 - The *IP_or_CIDR* clause is an IPv6 CIDR block that contains the IP address of the endpoint.

3.5 Matching an attribute clause

The information for the endpoint matches an attribute clause listed in a **with_attribute_list** clause if, when considering the name and value in that attribute clause, all of the following are true:

- If the name is **protocol**, then the protocol to be used for the network connection matches the value.
- If the name is **protocol.icmp.type**, then the ICMP type to be used for the network connection matches the value.
- If the name is **protocol.icmp.code**, then the ICMP code to be used for the network connection matches the value.
- If the name is **connection-state**, then all of the following are true:
 - If the value is **stateful**, then the network connection is to be stateful.
 - If the value is **stateless**, then the network connection is to be stateless.

4 Complete ZPR PEL Grammar, Expressed as BNF Productions

Nonterminal symbols (representing defined syntactic units) are in *italics* and enclosed in angle brackets $\langle \rangle$. Terminal symbols (that appear literally in actual policy statements) are in the code font. Alternative choices are separated by a vertical bar ‘|’ and for clarity are also displayed on separate lines.

The grammar below assumes all letters are lowercase. In practice, ZPR PELv1 is case-insensitive; all letters are first mapped to their lowercase equivalents. For example, the keywords `allow` and `ALLOW` and `Allow` and `aLLoW` are all considered equivalent. Keys and values are also case-insensitive.

```

<policy_statement> ::= <network_scope> <command> <source_endpoint> to <verb> <target_endpoint>
<command> ::= allow
<verb> ::= connect to
<source_endpoint> ::= <endpoint>
<target_endpoint> ::= <endpoint>
<network_scope> ::= in <security_attribute> vcn
<endpoint> ::= <basic_endpoint>
                | <basic_endpoint> <with_attribute_list>
<basic_endpoint> ::= <security_attribute> endpoints
                | <ip_or_CIDR>
                | all-endpoints
                | osn-services-ip-addresses
<with_attribute_list> ::= with <attribute>
                | <with_attribute_list> <list_conjunction> <attribute>
<list_conjunction> ::= ,
                | and
                | , and
<security_attribute> ::= <security_attribute_namespace> . <simple_key> : <simple_value>
                | <simple_key> : <simple_value>
                | '<security_attribute_namespace> . <simple_key> : <value>'
                | '<simple_key> : <value>'
<security_attribute_namespace> ::= <simple_name>
<simple_key> ::= <simple_name>
<simple_value> ::= <simple_name>
<simple_name> ::= any nonempty sequence of ASCII letters, digits, hyphens, and underscores
                that begins with a letter and whose length is not more than 255
<value> ::= <string>
<string> ::= any sequence of characters from the set of allowed string characters,
                which includes most “printable” Unicode characters; any apostrophe (single quote)
                character in the sequence should be escaped by writing two apostrophes
<attribute> ::= protocol = '<protocol_and_optional_port>'
                | protocol.icmp.type = '<icmp_type>'
                | protocol.icmp.code = '<icmp_code>'
                | connection-state = '<stateless_or_stateful>'
<protocol_and_optional_port> ::= <protocol>
                | <protocol>/<port>
<protocol> ::= <string>
<port> ::= <number>
                | <number> - <number>
<icmp_type> ::= <value>
<icmp_code> ::= <value>
<stateless_or_stateful> ::= stateless

```

| `stateful`

$\langle number \rangle ::=$ any nonempty sequence of ASCII digits

$\langle ip_or_CIDR \rangle ::=$ any valid IPv4 address or IPv6 address or IPv4 CIDR range or IPv6 CIDR range

5 Possible Future Directions for the OCI Policy Enforcement Language

5.1 Amalgamation of networks

A future version of Oracle ZPR PEL might provide for communication among multiple VCNs, each governed by its own set of policy enforcement statements, with communication between endpoints in different VCNs governed by an enforcement policy derived by combining the individual policy sets.

5.2 Policy enforcement within network routers

A future version of Oracle ZPR PEL might provide for enforcement of communication policy not just at connection endpoints but at intermediate points within the network.

5.3 Integration with other policy enforcement languages

A future version of Oracle ZPR PEL might support additional kinds of endpoints in order to express security policies that can also limit communication in other ways or according to criteria other than network endpoint attributes. For example, an endpoint might be a specific user or a specific service, and any given network endpoint might be associated with multiple users or multiple services, or both. Such an extended policy enforcement language might serve as a target language for a more abstract policy language, such as a similarly extended version of ZPL.